**Research Report — Task 2: Adding a PLC Code-Generation & Verification Loop**

**Topic:** *LLM-assisted PLC code generation and industrial automation programming*

**Base repository:** `catDforD/LLM_MAS_AutoGen` (original `advanced.py`)

---

# 1 Improvement Idea

The original MAS is a pure **research assistant**: it reads literature and writes a report, but it never *does* anything with the topic's domain output. Since the chosen topic is *LLM-assisted PLC code generation*, the most natural improvement is to make the MAS **actually generate and verify PLC code** rather than only describe how such systems work.

The improvement adds a **second stage** after the research stage:

```
[ research stage (reused from advanced.py) ]
  PlannerAgent -> SearchAgent -> WriterAgent -> ReviewerAgent  (<= 1 revision)

[ code stage (NEW) ]
  CodeGeneratorAgent -> CodeVerifierAgent
                        |  calls check_st_code (deterministic tool)
                        |-- FAIL -> back to CodeGeneratorAgent (<= 1 revision)
                        `-- PASS -> stop
```

This satisfies all Task 2 acceptance requirements:

1. **Not identical to the original architecture** — two new agent roles and a new workflow stage

are added.

1. **At least one real tool call whose result is used later** — `CodeVerifierAgent` calls the

deterministic `check_st_code` tool, and the returned `FAIL`/`PASS` drives the revision decision.

1. **Run on the same topic** — the improved MAS is run with the same PLC topic as Task 1.
2. **Comparison with the original** — see Section 6.

## 2 New Components

### 2.1 `CodeGeneratorAgent` (new role)

Generates a demo IEC 61131-3 Structured Text (ST) program from the research findings: a motor start/stop control with an emergency-stop safety interlock and a `TON` delay timer. It is told to wrap the code in a "‘ ‘`st` " `block and that ST assignment uses` := `(not` =‘).

### 2.2 `CodeVerifierAgent` (new role)

Calls `check_st_code` on the generated code and reports the tool's verdict. Like the original `SearchAgent`, it sets `reflect_on_tool_use=False` so the tool result is forwarded deterministically rather than through a model reflection step (which DeepSeek does not perform reliably after a tool call).

### 2.3 `check_st_code` (new deterministic tool, in `plc_tools.py`)

A *real* tool that performs no LLM call — it runs regex/counting checks and returns a `PASS`/`FAIL` report. After stripping comments (block `(* ... *)` and line `//`) it checks:

1. **Keyword pairing balance** — `IF`/`END_IF`, `CASE`/`END_CASE`, `FOR`/`END_FOR`,

`WHILE`/`END_WHILE`, `REPEAT`/`END_REPEAT`, `PROGRAM`/`END_PROGRAM`, etc.

1. **Bracket balance** — `()` and `[]`.
2. **Assignment-operator misuse** — flags a bare `=` where ST requires `:=` (the single most

common mistake LLMs make when emitting ST), while skipping condition lines and comparisons. A basic-structure check rejects inputs that contain no `PROGRAM`/`FUNCTION_BLOCK` declaration, so non-code text cannot pass vacuously.

### 2.4 Deterministic routing

The code stage uses a `SelectorGroupChat` with a custom `choose_code_next` router:

- `CodeGeneratorAgent` message $\rightarrow$ route to `CodeVerifierAgent`.
- `CodeVerifierAgent` message starting with `FAIL` (and revision budget left) $\rightarrow$ route back to

CodeGeneratorAgent, incrementing the revision counter.

- otherwise $\rightarrow$ stop; the team terminates on the deterministic PASS text.

Because the routing inspects the *tool result* directly (not a fragile model reflection), the revision loop is reproducible.

## 3 Implementation

Two new files were added; the original files (basic.py, advanced.py, common.py, research_tools.py) were left unchanged:

- plc_tools.py — the check_st_code tool and comment-stripping helpers.
- improved.py — the two-stage improved MAS, which imports build_team from advanced.py for the

research stage and defines the code stage above.

## 4 Run Results

improved.py was run with the same PLC topic. Outputs go to outputs/improved_<timestamp>/:

- report.md — the research report (produced by the reused research stage; 1 revision occurred).
- review.md — the reviewer's feedback plus the citation audit.
- generated_code.st — the final, verified ST program (62 lines, see

screenshots/08_improved_generated_code.png).

- verification.md — the check_st_code result and a deterministic re-check, both PASS

(screenshots/09_improved_verification.png).

- trace.jsonl — the full trace of both stages.
- run_summary.json — research_revisions: 1, code_revisions: 0, code_verification_pass: true.

The generated code declares PROGRAM MotorControl \ldots{} END_PROGRAM, uses IF/ELSIF/END_IF, instantiates a TON timer, and uses := for every assignment. The verifier confirmed balanced keywords and brackets and no assignment-operator misuse.

## 5 Tool-Call Analysis

The trace shows the code stage's tool loop: `CodeGeneratorAgent` emits a message containing a

```
with the code as the `code` argument, followed by a `ToolCallExecutionEvent` whose result is
`PASS`. That result is *used later* in two ways: (a) it is the routing signal that ends the l
and (b) it is recorded in `verification.md` as the verification evidence.

(When the tool returns `FAIL` --- e.g. an unbalanced `IF`/`END_IF` or a bare `=` --- the rout
the code back to `CodeGeneratorAgent` for a revision; the earlier debugging runs exercised th
path before the checker's comment-handling was finalized.)

## 6. Comparison with the Original MAS

| Dimension | Original (`advanced.py`) | Improved (`improved.py`) |
|---|---|---|
| Roles | Planner, Search, Writer, Reviewer | + CodeGenerator, CodeVerifier |
| Output | cited research report only | research report **+ verified ST code** |
| Tools | `search_literature`, `audit_citations` | + `check_st_code` (deterministic) |
| Workflow | 1 revision loop (research) | research loop + code generate--verify loop |
| Reliability mechanism | citation audit, max turns | + deterministic code verification |
| Relevance to topic | describes PLC-code MAS | also produces and checks PLC code |

The improvement trades a small amount of additional runtime for a much more concrete, on-topi
demonstration: instead of only *describing* how an LLM MAS can generate PLC code, the improve
**does it and verifies the result with a real tool**. The verification is deterministic (no
reliance on the LLM to judge its own output), which is exactly the kind of reliability mechan
the course emphasizes for industrial-control code.

## 7. Conclusion

Task 2 was completed: the original MAS was extended with a PLC code-generation and verificati
stage (two new roles, one new deterministic tool, and a new generate--verify--revise loop), i
run on the same research topic, and its run results were compared with the original. Screensh
and the generated/verified code are included in `/screenshots`.
```